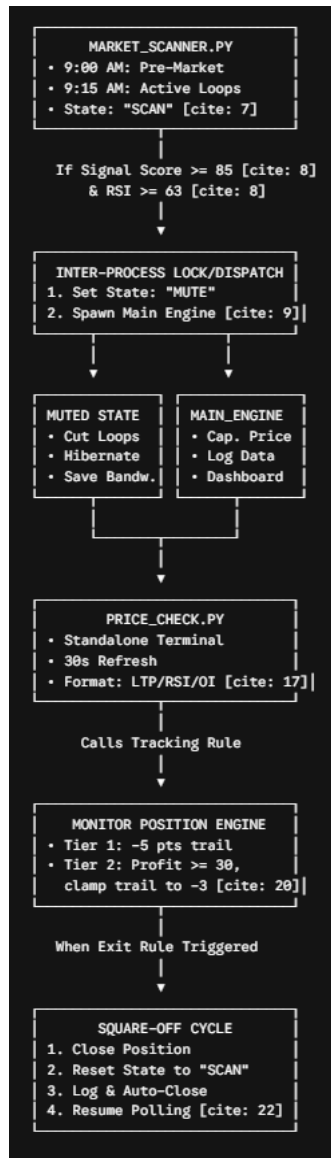


This design features a **decoupled, multi-process architecture**. By separating the high-frequency scanning engine from the tracking dashboards, the core script never experiences delay lags while communicating with broker APIs.



Architectural Strategy Review

1. Isolated State Pipelining (Inter-Process Communication)

Instead of running inside a single python application script—where a crash or network slowdown in your logging window could take down your primary execution flow—the workspace is divided across three separate execution spaces:

- **The Sentry (Market_Scanner.py):** Runs continuously, watching for clean momentum breakouts.
- **The Executioner (main_execution_engine.py):** Spawns strictly to manage risks and execute exits.
- **The Tracker (Price_Check.py):** Spawns strictly to print updates to your visual desktop console.

2. Micro-Overhead Mute Logic

By establishing scanner_state.json as a localized cross-process bridge, the scanner changes behavior instantly when a position goes live. Because it completely stops polling data frames during active trades, **your broker API traffic drops significantly**. This prevents rate-limiting issues and guarantees full network bandwidth is dedicated exclusively to the tracking engine for fast execution when the stop-loss hits.

3. Automated Re-Arming Circuit

The main execution engine owns the state lock. The moment it confirms an exit, it updates the state file back to "SCAN" and exits. The scanner immediately wakes up from hibernation and continues monitoring without requiring you to restart any code manually.

Plaintext



```
=====
Traded Instrument : NIFTY 28th May 24200 CE
Current LTP      : Rs 142.50
Current OI       : Increased from Order Execution by 14.2%
Current RSI      : 67.42 (Looking at the 1 min Chart)
Status          : HOLD / Neutral
=====
```

FINAL FILE STRUCTURE

project/

|

├─ main.py

├─ strategy_engine.py

├─ risk_event_engine.py

├─ monitor_position_engine.py

|

└─ logs/